

CYLAB SECURITY ACADEMY

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	Evallo, Daniella Kim N.
Section / Class	BSIT-3G
Challenge Name	Password Profiler 2
Category	General Skills
Points / Difficulty	Hard
Date Solved	08/14/2026
Flag Format Given	Given PicoCTF{...}

1. Challenge Description

The challenge involved connecting to an SMB server and interacting with a public file share. The goal was to identify the correct SMB port, access the available share, inspect the provided Bash script and log file, upload the script to the share, wait for the scheduled process to execute, and retrieve the updated log file containing the flag.

2. Initial Analysis / Reconnaissance

I first tested the target server and checked which port was available for the SMB service. The initial connection attempt using port 55673 failed, while port 52542 was open. I then used smbclient to enumerate the available SMB shares. The server revealed three shares: shares, secure-shares, and IPC\$. The shares directory was accessible without a username or password, so I used it for further investigation.

3. Tools and Commands Used

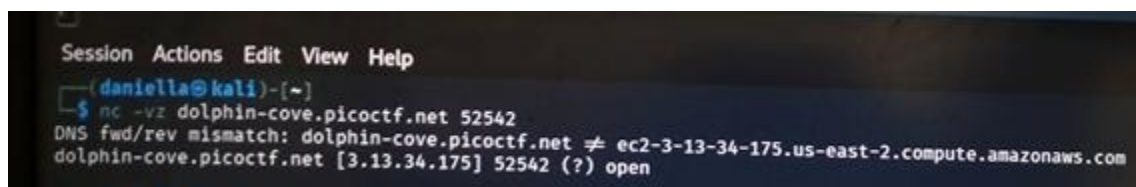
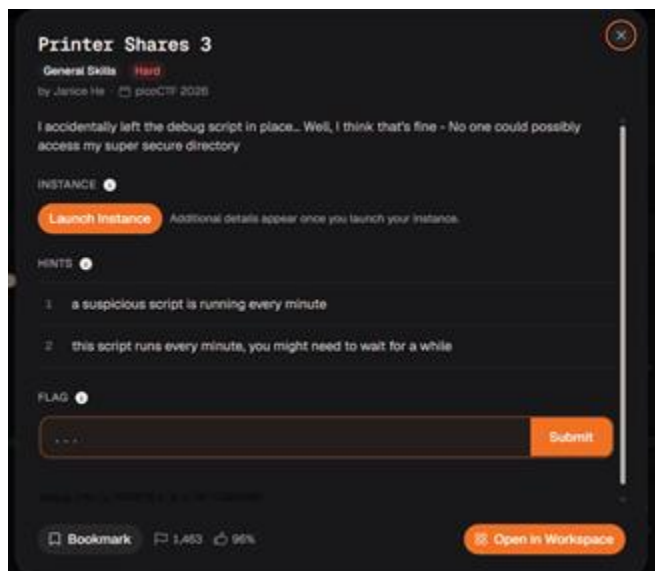
List every tool/command you used, in the order you used them, and what each one told you.

Tool / Command	What it was used for	Result / Output
nc -vz dolphin-cove.picoctf.net 52542	Check whether the target port was open.	Port 52542 was open.
smbclient -L //dolphin-cove.picoctf.net -p 52542 -N	List the available SMB shares.	Found shares, secure-shares, and IPC\$.

Tool / Command	What it was used for	Result / Output
smbclient //dolphin-cove.picoctf.net/shares -p 52542 -N	Connect to the public shares directory.	Successfully connected.
ls	List files inside the SMB share.	Found script.sh and cron.log.
get script.sh	Download the Bash script for inspection.	Retrieved script.sh.
get cron.log	Download the log file.	Retrieved cron.log.
cat script.sh	Read the contents of the Bash script.	The script printed a health-check message with the current date.
smbclient //dolphin-cove.picoctf.net/shares -p 52542 -N -c "put script.sh"	Upload the script to the SMB share.	script.sh was successfully uploaded.
sleep 65	Wait for the scheduled process to run.	Allowed the cron job to execute.
smbclient //dolphin-cove.picoctf.net/shares -p 52542 -N -c "get cron.log"	Retrieve the updated log.	Downloaded the updated cron.log.
cat cron.log	Read the generated log.	The flag appeared in the output.

4. Solution Walkthrough

Step 1



I first checked whether the target SMB port was accessible by running:

nc -vz dolphin-cove.picocftf.net 52542

The terminal showed that the connection to port 52542 was open, confirming that this was the correct port to use.

Step 2

then listed the SMB shares using:

smbclient -L //dolphin-cove.picocftf.net -p 52542 -N

The server returned the following shares:

ShareName Type

----- ----

shares Disk

secure-shares Disk

IPC\$ IPC

The shares directory was identified as the public share.

```
(daniella@kali)-[~]
$ smbclient -L //dolphin-cove.picocftf.net -p 52542 -N

Sharename      Type           Comment
-----
shares         Disk          Public Share With Guests
secure-shares  Disk          Printer for internal usage only
IPC$           IPC           IPC Service (Samba 4.19.5-Ubuntu)
Reconnecting with SMB1 for workgroup listing.
do_connect: Connection to dolphin-cove.picocftf.net failed (Error NT_STATUS_IO_TIMEOUT)
Unable to connect with SMB1 -- no workgroup available
```

Step 3

I connected to the public share using:

smbclient //dolphin-cove.picocftf.net/shares -p 52542 -N

After connecting, I used:

smb: \> ls

The directory contained script.sh and cron.log.

```
(daniella@kali)-[~]
$ smbclient //dolphin-cove.picocftf.net/shares -p 52542 -N
Try "help" to get a list of possible commands.
smb: \> ls

.                D          0  Fri Aug 14 21:05:01 2026
..               D          0  Fri Aug 14 21:05:01 2026
script.sh        N         73  Thu Feb  5 05:22:17 2026
cron.log         N         86  Fri Aug 14 21:06:01 2026
```

Step 4

I downloaded the existing files from the SMB share:

```
get script.sh
get cron.log
```

I then exited the SMB client using:

```
Smb: \> exit
```

After downloading the script, I inspected it using:

```
cat script.sh
```

The script contained:

```
#!/bin/bash
# this script runs every minute
echo "Health Check: $(date)"
```



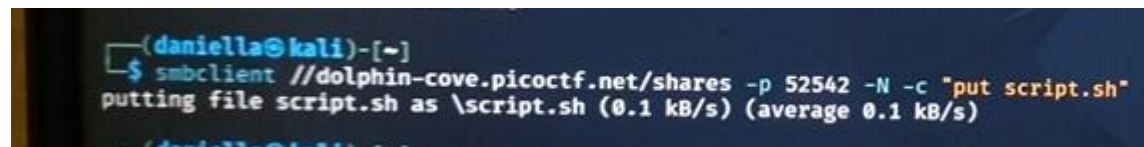
```
65536 blocks of size 1024. 57340 blocks available
smb: \> get script.sh
getting file \script.sh of size 73 as script.sh (0.1 KiloBytes/sec) (average 0.1 KiloBytes/sec)
smb: \> get cron.log
getting file \cron.log of size 86 as cron.log (0.1 KiloBytes/sec) (average 0.1 KiloBytes/sec)
smb: \> exit

(daniella@kali)-[~]
$ cat script.sh
#!/bin/bash
# this script runs every minute
echo "Health Check: $(date)"
```

Step 5

```
smbclient //dolphin-cove.picoc.tf.net/shares -p 52542 -N -c "put script.sh"
```

The upload succeeded and the terminal showed that script.sh was put on the share.



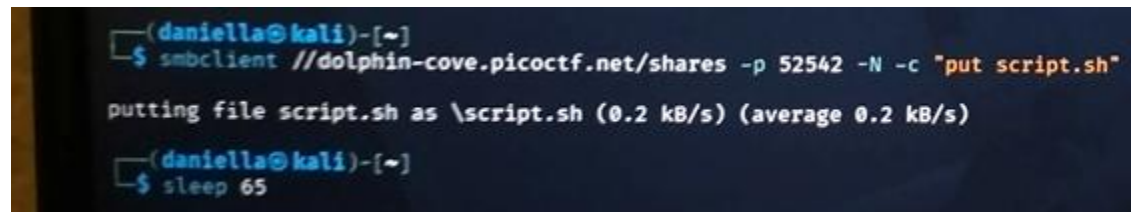
```
(daniella@kali)-[~]
$ smbclient //dolphin-cove.picoc.tf.net/shares -p 52542 -N -c "put script.sh"
putting file script.sh as \script.sh (0.1 kB/s) (average 0.1 kB/s)

(daniella@kali)-[~]
```

Step 6

I waited for the scheduled process to execute by running: **sleep 65**

The wait was slightly longer than one minute because the script was configured to run every minute.



```
(daniella@kali)-[~]
$ smbclient //dolphin-cove.picoc.tf.net/shares -p 52542 -N -c "put script.sh"
putting file script.sh as \script.sh (0.2 kB/s) (average 0.2 kB/s)

(daniella@kali)-[~]
$ sleep 65
```

Step 7

After waiting, I downloaded the updated log: **smbclient //dolphin-cove.picoc.tf.net/shares -p 52542 -N -c "get cron.log"**

then displayed its contents using: **cat cron.log**

The log contained the normal health-check entries as well as the challenge flag.

```
(daniella@kali)-[~]
$ smbclient //dolphin-cove.picoctf.net/shares -p 52542 -N -c "get cron.log"
getting file \cron.log of size 610 as cron.log (0.6 KiloBytes/sec) (average 0.6 KiloBytes/sec)

(daniella@kali)-[~]
$ cat cron.log
Health Check: Fri Aug 14 13:05:01 UTC 2026
Health Check: Fri Aug 14 13:06:01 UTC 2026
Health Check: Fri Aug 14 13:07:01 UTC 2026
Health Check: Fri Aug 14 13:08:01 UTC 2026
Health Check: Fri Aug 14 13:09:01 UTC 2026
```

Step 8

The important output from the log was:

picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}

This confirmed that the challenge had been successfully solved

```
(daniella@kali)-[~]
$ smbclient //dolphin-cove.picoctf.net/shares -p 52542 -N -c "get cron.log"
getting file \cron.log of size 610 as cron.log (0.6 KiloBytes/sec) (average 0.6 KiloBytes/sec)

(daniella@kali)-[~]
$ cat cron.log
Health Check: Fri Aug 14 13:05:01 UTC 2026
Health Check: Fri Aug 14 13:06:01 UTC 2026
Health Check: Fri Aug 14 13:07:01 UTC 2026
Health Check: Fri Aug 14 13:08:01 UTC 2026
Health Check: Fri Aug 14 13:09:01 UTC 2026
Health Check: Fri Aug 14 13:10:01 UTC 2026
Health Check: Fri Aug 14 13:11:01 UTC 2026
picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}
Health Check: Fri Aug 14 13:12:01 UTC 2026
picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}
Health Check: Fri Aug 14 13:13:01 UTC 2026
picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}
Health Check: Fri Aug 14 13:14:01 UTC 2026
picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}
```

5. Flag

picoCTF{5mb_pr1nter_5h4re5_r3v3r53_7c819679}

6. Key Concept / What I Learned

This challenge demonstrated how SMB shares can expose files that are used by scheduled processes by enumerating the SMB service, identifying an accessible share, examining the available script and log files, and observing the scheduled execution of the script, it was very hard but I was able to understand how the challenge worked. The main lesson is that writable network shares and scheduled scripts can create security risks when they are not properly restricted.

7. References

[Printer Shares 3 — picoCTF Writeup | by mayhack | Medium](#)